

Magnitude-Ordered Visibility Graphs for Interpretable Time Series Feature Extraction

a dissertation submitted for the degree of
Electronic and Computer Engineering

Sahas Talasila

Student number: 230057896

May 2026

Word count:

CONTENTS

Self-reflection and feedback	3
Engineering skills portfolio indexing	4
1 Introduction	6
2 Literature Review and Motivation	7
2.1 Time Series Feature Extraction	7
2.2 Visibility Graphs for Time Series	7
2.3 Path Signatures in Machine Learning	8
2.4 The Interpretability Gap	8
3 Theoretical Background	9
3.1 MOVG Construction	10
3.1.1 Visibility Condition	10
3.1.2 Magnitude Hierarchy and Incremental Insertion	10
3.1.3 Worked Example	11
3.2 Path Decomposition	13
3.2.1 Density and Chronological Constraints	13
3.2.2 Chain Discovery	13
3.2.3 Worked Example (continued)	14
3.3 Path Signatures	15
3.3.1 The 1D Degeneracy	15
3.3.2 2D Embedding and Signature Definition	15
3.3.3 Extracted Scalar Projections	16
3.3.4 Aggregate and Expanded Signatures	16
3.3.5 Worked Example (continued)	16
3.4 Baseline Preprocessing Methods	17
3.4.1 Fast Fourier Transform (FFT)	17
3.4.2 Wavelet Decomposition	17
4 Methodology	18
4.1 Project Deliverables	18
4.2 Feature Extraction Pipeline	18
4.2.1 Feature Reduction Pipeline	18
4.2.2 Baseline Feature Sets	19
4.2.3 Windowed Construction and Multi-Scale Capture	19
4.3 Models and Evaluation	20
4.3.1 One-Step-Ahead Prediction	20
4.3.2 Walk-Forward Cross-Validation	20
4.4 Post-Inference Backtrace	21
4.4.1 Limitations of SHAP for Structural Features	22
5 Results	22
5.1 Evaluation protocol	22
5.2 Cross-dataset comparison	25
5.3 Per-dataset and per-model breakdown	25

5.4	Feature reduction	26
5.5	Feature importance	26
5.6	Backtracing demonstration	26
6	Discussion	27
6.1	Interpretability: from prediction to chain to time index	28
6.2	Cross-domain generalisation	28
6.3	Model-by-model behaviour	29
6.4	Comparison to baseline preprocessing	29
6.5	Honest caveats	29
6.6	Synthesis	29
7	Conclusion	30
8	Constraints and Future Work	30
8.1	Implementation constraints	30
8.2	Methodological constraints	31
8.3	Risk management and mitigation	31
8.4	Intellectual property and data management	31
8.5	Ethical considerations and EDI	32
8.6	Future work	32
9	Appendix	34
9.1	Images	34
9.2	Reduced Feature Set	39
9.3	Per-Dataset RMSE Tables	39

SELF-REFLECTION AND FEEDBACK

Complete this section so that your examiners can provide tailored feedback to your work (up to 50 words per answer). This is a reflective narrative for feedback purposes only; your marks will not be informed by information on this page.

I believe a strong part of my submission is:

The mechanistic interpretability chain. Predictions trace deterministically through SHAP attribution, MOVG chains, and graph nodes back to specific timestamps in the raw signal. This is a property the FFT and wavelet baselines cannot offer, and it underpins the central methodological contribution of the work.

An area I struggled with is:

Computational performance of the visibility-check stage. The $\mathcal{O}(n^2)$ construction cost forced a sliding-window evaluation strategy and limited the series lengths I could realistically benchmark on. A GPU-resident reimplementaion was outside the project scope, and the dependency on **NetworkX** constrained parallelisation options.

I would particularly like feedback/help with:

The strength of the cross-domain generalisation claim. MOVG records the lowest mean RMSE on every one of the seven datasets evaluated, but the datasets are all univariate and span only four distinct signal regimes. Feedback on whether the evidence justifies the generality claim made in the Discussion would be valuable.

ENGINEERING SKILLS PORTFOLIO INDEXING

Your dissertation needs to demonstrate you have achieved certain learning outcomes as a result of your work which you must reflect on. Please use this checklist to index the section number(s) which demonstrate the appropriate skill. **This page must be filled out** and included in the final report to pass the project.

Engineering skill	Description	Section(s)
Environmental and societal impact	Where in your report have you demonstrated that you have considered the wider impact on society to the broader area that you have been working in?	§1, §6.5
Ethical considerations	All projects contain ethical considerations. Have you researched any potential negative impacts of your project? Are there ethical questions in literature or journalism about the area you are working in? What is the general consensus of the engineering community of these issues?	§2.4, §8.5
Risk management and mitigation	Have you planned for unexpected outcomes or circumstances? How did you manage changes to your project? Did you decide to change your project and if so how did it change the risk profile of the work?	§4.1, §8.3
Intellectual property, security and data management	How have you ensured that the data or intellectual property (IP) which you have generated within your project is secure from external actors or accidental loss? Where have you demonstrated an awareness of how IP is captured? How was information management included in part of your project planning?	§4.1, §8.4
Equality, diversity and inclusion	Where have you considered the inclusivity of your work? Have you demonstrated that you have considered how your work may impact different groups? Is your work accessible to a wide range of users and could others from a different background to yourself repeat your results based on what you have written in your report?	§4.1, §8.5
Quality of information	Where in your report have you discussed the <i>quality</i> of the information you have researched or the data you have presented? Have you reflected on how this could be improved?	§4.1, §5.1, §6.5
Continual professional development and self-reflection	Where in your report have you reflected on your own performance throughout the project? Have you identified personal strengths and achievements as well as weaknesses? Have you made plans for exploiting the strengths and improving the weaknesses?	§6.5, §7, §8

Abstract

Time series preprocessing methods typically discard the structural link between extracted features and specific events in the input signal. This report presents the Magnitude-Ordered Visibility Graph (MOVG), a framework that converts a time series into a directed acyclic hierarchy by combining geometric visibility checks with magnitude-based ordering. The hierarchy is decomposed into monotone chains, from which path-signature projections and structural descriptors produce a fixed-length feature vector. Every feature is traceable through a specific chain and set of graph nodes back to identifiable timestamps in the raw signal. When combined with SHAP attribution, this provenance enables a complete backtrace from model prediction to raw data.

The framework was formally specified, implemented as a Python package with 13 modules and 45 unit tests, and evaluated against FFT and wavelet preprocessing across seven datasets and six regression models. MOVG recorded the lowest mean RMSE on every dataset, with an average improvement of 15.1% over the best non-MOVG baseline. A 32-feature reduced representation retained this performance advantage while supporting deterministic post-inference backtracing.

INTRODUCTION

Accurate time series forecasting underpins decision-making across engineering, finance, and environmental monitoring. Current preprocessing methods, including spectral transforms and statistical feature toolkits, can achieve strong predictive performance but typically produce features that cannot be linked back to specific events in the input signal. When a model makes a prediction, the practitioner can identify which features mattered but not which part of the raw data produced those features. This opacity limits trust and actionability in safety-critical and high-stakes applications.

This project addresses the interpretability gap by developing a novel feature extraction framework called the Magnitude-Ordered Visibility Graph (MOVG). MOVG constructs a directed acyclic graph from a time series by combining pairwise visibility checks with a magnitude-based hierarchy. The graph is decomposed into monotone chains, and path signatures computed over these chains produce fixed-dimensional descriptors. Each descriptor is traceable through a specific chain, through specific graph nodes, to specific timestamps and amplitudes in the original signal.

The project produced three deliverables:

1. A formal algorithmic specification of the MOVG framework, including definitions, proofs of acyclicity, pseudocode, and an installable Python package (13 modules, `pyproject.toml`, 45 unit tests).
2. API documentation for the package in RESTful style, enabling future researchers to reproduce and extend the work.
3. A preprocessing benchmark comparing MOVG features against FFT and wavelet baselines across seven datasets and six regression models, together with a post-inference backtracing tool that maps predictions to raw signal events.

The report is structured as follows. Section 2 reviews existing feature extraction methods and identifies the interpretability gap. Section 3 presents the theoretical background of

the MOVG construction. Section 4 describes the experimental methodology. Section 5 reports the empirical results. Section 6 interprets the findings. Section 7 summarises the contributions, and Section 8 discusses limitations and future work.

LITERATURE REVIEW AND MOTIVATION

2.1 Time Series Feature Extraction

Raw time series are high-dimensional, temporally correlated, and often noisy. Although models can ingest sequences directly, feature preprocessing usually improves accuracy and efficiency. The preprocessing choice determines which information is preserved.

Spectral methods project signals into the frequency domain. The fast Fourier transform decomposes signals into sinusoidal components and reveals dominant periodicities [9, 12]. Recent work combines FFT amplitude and phase features with LSTMs to leverage time- and frequency-domain information [10]. Transformer variants using FFT-based preprocessing improve forecasting across sensor datasets by enriching representations with spectral structure [11].

Wavelet and modal methods provide joint time-frequency localisation [8]. MDCNet uses variational mode decomposition to separate trend and modal components and achieves strong long-horizon performance [23]. These approaches characterise frequency content or scale-specific energy but do not encode structural relationships between temporal events.

Statistical feature extraction remains in the time domain. Toolkits such as *hctsa* compute large collections of scalar summaries including distributional statistics, autocorrelation measures, and entropy estimates. Deep learning models instead learn representations end-to-end from raw sequences. THATSN builds a temporal hierarchical tree from multi-cyclic decompositions and aggregates features using a gated LSTM to capture inter-cycle relationships [24]. Despite strong predictive performance, these learned representations are typically opaque.

Across both paradigms, the mapping from raw signal to feature is rarely transparent. Spectral coefficients, statistical summaries, and learned embeddings may support accurate predictions, but they do not provide a clear link between predictions and specific events in the input signal.

2.2 Visibility Graphs for Time Series

Visibility graphs map geometric relationships in time series to graphs: two points connect if no intermediate point blocks their line of sight [1]. This preserves temporal ordering and reflects underlying dynamics, since degree distributions distinguish periodic, random, and fractal processes [1]. Extensions include multiplex visibility graphs, where each channel forms a layer and inter-layer edges capture cross-variable dependencies [3, 4]. Broader overviews cover classification, clustering, and motif discovery [5], while applications to financial and synthetic data demonstrate extraction of short- and long-term structure [7].

Standard visibility graphs still have structural limitations. They are undirected, contain cycles, and lack a canonical traversal order. Systematic decomposition into interpretable substructures becomes computationally difficult. As a result, they are limited as preprocessing for machine learning. Graph-level statistics such as degree distribution or

clustering coefficient can be used as features, but they cannot be linked to specific events in the original signal.

Figure 1 shows an example of a standard visibility graph constructed from a time series. The undirected edges reflect mutual visibility, but the presence of cycles and lack of hierarchy means that there is no straightforward way to decompose the graph into meaningful substructures for feature extraction. This motivates the need for a new approach that combines visibility with a magnitude-based hierarchy to produce a directed acyclic structure amenable to systematic decomposition and traceable feature extraction.

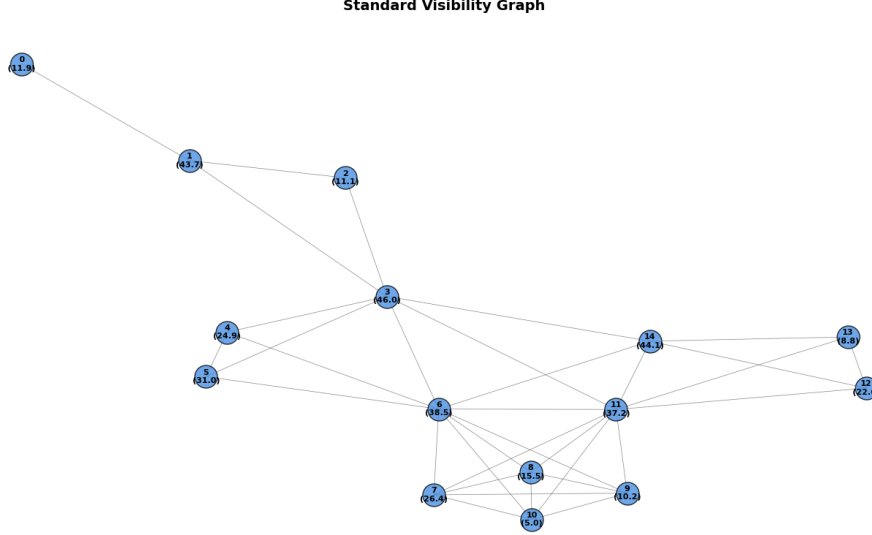


Figure 1: Example Visibility Graph

2.3 Path Signatures in Machine Learning

The signature transform, rooted in rough path theory, maps a sequential path to a graded sequence of iterated integrals that uniquely characterises it up to reparametrisation [2]. Linear functionals on signature terms are dense in the space of continuous functions on paths, providing a universal nonlinear feature map. The output dimensionality depends only on the embedding dimension and truncation depth, not solely on input length, a property well-suited to variable-length sequences.

A generalised signature framework has been developed and benchmarked across 26 multivariate classification tasks, achieving performance comparable to deep recurrent and convolutional architectures [2]. Applications span handwriting recognition, medical time series, and financial modelling. However, existing work applies signatures to raw or augmented time series. No prior method computes signatures over structurally decomposed paths derived from a graph-based representation of the signal.

2.4 The Interpretability Gap

The methods surveyed above share a common structural deficiency: the chain of reasoning from raw signal to extracted feature to model prediction is opaque. FFT projects into the frequency domain, discarding temporal locality. Wavelet decomposition retains

time-frequency resolution but commits to a fixed basis, and individual coefficients do not correspond to identifiable signal events. Standard visibility graphs encode pairwise geometric relationships but lack the directed, acyclic structure required for systematic decomposition. Deep learning methods learn representations end-to-end with no human-readable intermediate stage.

Post-hoc explainability tools such as SHAP can identify *which* features drive a prediction. However, the explanatory value of a SHAP attribution depends entirely on the semantic content of the underlying feature. An attribution to a spectral coefficient or an autocorrelation statistic tells the practitioner that a particular frequency or correlation property mattered. It does not reveal which events in the raw signal produced that property.

This work addresses the gap by proposing a preprocessing method in which every extracted feature possesses full structural provenance. The Magnitude-Ordered Visibility Graph transforms a time series into an acyclic hierarchy admitting polynomial-time decomposition into monotone chains. Path signatures computed over these chains yield fixed-dimensional descriptors, each traceable through a specific chain, through specific graph nodes, to specific timestamps and amplitudes in the original signal. When combined with post-hoc attribution, this provenance enables a complete backtrace: from model prediction, through feature importance, through graph structure, to the raw data.

THEORETICAL BACKGROUND

Two existing approaches motivate the design of MOVG. Standard visibility graphs, introduced by Lacasa et al. [1], convert a time series into an undirected graph by connecting any two points that have unobstructed line-of-sight. Because visibility is symmetric, cycles arise immediately: any three mutually visible points form a triangle. Consequently, no canonical traversal order exists, path enumeration grows exponentially, and the resulting graph cannot be systematically decomposed for feature extraction.

Min-max trees offer an alternative hierarchical view [17, 18]. A min-max tree recursively splits a time series at its global extremum, forming left and right subtrees. This produces a binary hierarchy that encodes dominant peaks at shallow levels and minor fluctuations at depth. However, the recursive bisection is entirely determined by index ranges and ignores the geometric relationships between points. Two points may be placed in the same subtree despite having no line-of-sight connection, or separated into different subtrees despite being geometrically adjacent. The structure therefore does not reflect the actual shape of the signal.

MOVG combines the geometric grounding of visibility graphs with the hierarchical organisation of min-max trees, while avoiding the limitations of both. It does so through two conditions applied simultaneously during an incremental construction: a *visibility condition* that determines which connections are geometrically valid, and a *magnitude ordering* that determines the parent-child hierarchy. The result is a directed acyclic forest in which every edge reflects both a real geometric relationship in the signal and a dominance ranking by amplitude. Cycles are impossible because each parent-child step moves to strictly smaller amplitude.

3.1 MOVG Construction

MOVG is built incrementally. Each time series point is inserted in chronological order, and at each step the algorithm checks two conditions: whether the new point is visible to existing points, and which visible predecessor with larger amplitude should become its parent. The tree grows one node at a time, with the structure fully determined after the final insertion.

3.1.1 Visibility Condition

Two points i and j ($i < j$) in a time series $Y = \{y_0, y_1, \dots, y_{n-1}\}$ are mutually visible if and only if no intermediate point breaks the line of sight between them:

$$y_k \leq y_i + (y_j - y_i) \cdot \frac{k - i}{j - i} \quad \forall i < k < j \quad (1)$$

In plain terms: draw a straight line from (i, y_i) to (j, y_j) . If every point between them lies on or below that line, the two points can “see” each other. Adjacent points ($j = i + 1$) are always visible because no intermediate point exists to block the view.

3.1.2 Magnitude Hierarchy and Incremental Insertion

The tree is constructed by processing points one at a time in chronological order. When a new point n_i is inserted, the algorithm looks backward to all previously inserted points that are both *visible* to n_i and have *strictly larger amplitude*. Among these candidates, it selects the one with the **smallest** amplitude as parent:

$$\text{parent}(i) = \arg \min_{c \in C_i} Y[c], \quad C_i = \{j : j < i, \text{ visible}(j, i), Y[j] > Y[i]\} \quad (2)$$

If a newly inserted point has larger amplitude than every previously inserted point, it becomes the new root and the former root becomes its child. If the candidate set C_i is empty (no earlier visible point has larger amplitude), the node becomes a secondary root of a separate tree.

Selecting the *smallest* larger visible predecessor (rather than the largest) produces a tight hierarchy. This means each node attaches to the “closest ceiling” above it rather than jumping directly to the global maximum. The result is that root-level nodes represent dominant peaks, leaf nodes represent minor fluctuations, and depth directly encodes amplitude significance.

The resulting structure is a **forest**: a collection of one or more trees. Secondary roots arise when a point appears in the time series before any point with larger amplitude. These early-arriving peaks have no valid parent candidate and form the roots of their own subtrees. The forest is provably acyclic because every parent-child edge connects a node to a strictly smaller-amplitude child. Any hypothetical cycle would require returning to a node with larger amplitude than its parent, contradicting the construction rule.

Algorithm 1 Incremental MOVG Construction

Require: Time series Y of length n **Ensure:** Forest $\mathcal{F}$ (parent map, level map, root set)

```
1: function BUILDMOVG( $Y$ )
2:   parent[.]  $\leftarrow$  nil; level[.]  $\leftarrow$  0; root  $\leftarrow$  0
3:   for  $i \leftarrow 0$  to  $n - 1$  do ▷ Insert nodes chronologically
4:      $C \leftarrow \emptyset$ 
5:     for  $j \leftarrow 0$  to  $i - 1$  do ▷ Check all predecessors
6:       if ISVISIBLE( $Y, j, i$ ) and  $Y[j] > Y[i]$  then
7:          $C \leftarrow C \cup \{j\}$ 
8:       end if
9:     end for
10:    if  $C \neq \emptyset$  then
11:      parent[ $i$ ]  $\leftarrow \arg \min_{c \in C} Y[c]$  ▷ Tightest ceiling
12:      level[ $i$ ]  $\leftarrow$  level[parent[ $i$ ]] + 1
13:    else if  $Y[i] > Y[\text{root}]$  then
14:      Reparent former root under  $i$ ; root  $\leftarrow i$  ▷ New global max
15:    else
16:      Mark  $i$  as secondary root; level[ $i$ ]  $\leftarrow$  0
17:    end if
18:  end for
19:  return (root, parent, level)
20: end function
```

The visibility check for each pair requires examining up to $O(n)$ intermediate points. Over n insertions, each checking up to n predecessors, the worst-case complexity is $O(n^3)$. In practice, visibility checks terminate early when an obstruction is found, and typical performance is closer to $O(n^2)$ for signals with moderate local structure.

3.1.3 Worked Example

Consider the 8-sample time series $Y = [3, 1, 4, 2, 5, 1, 3, 2]$, plotted in Figure 2.

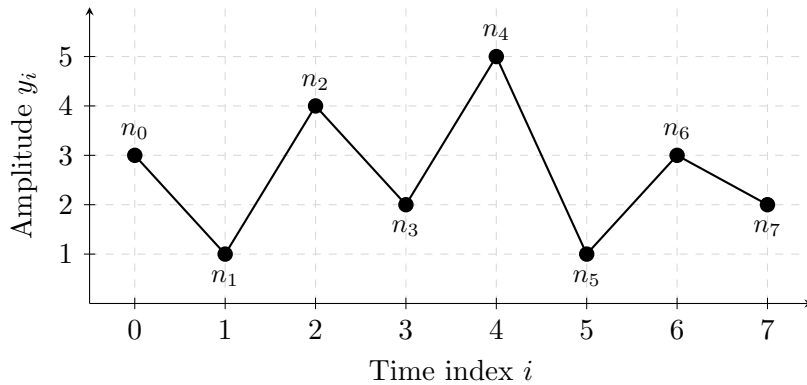


Figure 2: The 8-sample worked example $Y = [3, 1, 4, 2, 5, 1, 3, 2]$. Nodes are labelled $n_0, \dots, n_7$ in chronological order. Node n_4 ($y=5$) is the global maximum.

The tree is built by inserting each node in chronological order. At each step, the new

node looks backward for visible predecessors with strictly larger amplitude, then attaches to the one with the smallest amplitude (the tightest ceiling).

1. **Insert** n_0 ($y=3$). First node. No predecessors exist. n_0 becomes the root.
2. **Insert** n_1 ($y=1$). The only predecessor is n_0 ($y=3$), which is visible (adjacent) and has larger amplitude. n_1 attaches to n_0 as a child at depth 1.
3. **Insert** n_2 ($y=4$). Predecessors are n_0 ($y=3$) and n_1 ($y=1$). Neither has amplitude greater than 4, so no valid parent exists. However, n_2 has larger amplitude than the current root n_0 , so n_2 becomes the new root and n_0 is reparented as a child of n_2 .
4. **Insert** n_3 ($y=2$). Check visibility backward: n_2 ($y=4$) is adjacent and visible, with amplitude $4 > 2$. n_0 ($y=3$) has amplitude $3 > 2$ but is *not* visible to n_3 because n_2 ($y=4$) at index 2 sits above the line of sight from $(0, 3)$ to $(3, 2)$. The only valid candidate is n_2 . n_3 attaches to n_2 at depth 1.
5. **Insert** n_4 ($y=5$). No predecessor has amplitude greater than 5. n_4 becomes the new root. The previous root n_2 is reparented under n_4 (they are visible via the edge set).
6. **Insert** n_5 ($y=1$). The only visible predecessor with larger amplitude is n_4 ($y=5$), which is adjacent. n_5 attaches to n_4 at depth 1.
7. **Insert** n_6 ($y=3$). The visible predecessor with larger amplitude is n_4 ($y=5$). n_6 attaches to n_4 at depth 1.
8. **Insert** n_7 ($y=2$). Visible predecessors with larger amplitude include n_4 ($y=5$) and n_6 ($y=3$). The smallest is n_6 ($y=3$), the tightest ceiling. n_7 attaches to n_6 at depth 2.

The completed hierarchy is a forest with three trees. The primary tree is rooted at n_4 ($y=5$) with subtree n_2 ($y=4$) containing children n_0 and n_3 , and direct children n_5 and n_6 with grandchild n_7 . The forest mirrors the dominance structure of the signal: the global peak n_4 anchors the largest subtree. Secondary peaks that appeared before any larger node (n_0 , which arrived before n_2) are absorbed when a larger node is inserted. The depth of any node encodes its relative amplitude significance: shallow nodes correspond to dominant features, deep nodes to fine-scale fluctuations.

Figure 3, shown on the next page for clarity, depicts the fully built MOVG for this example. Each node is labelled with its index, and edges reflect the parent-child relationships established by the construction algorithm.

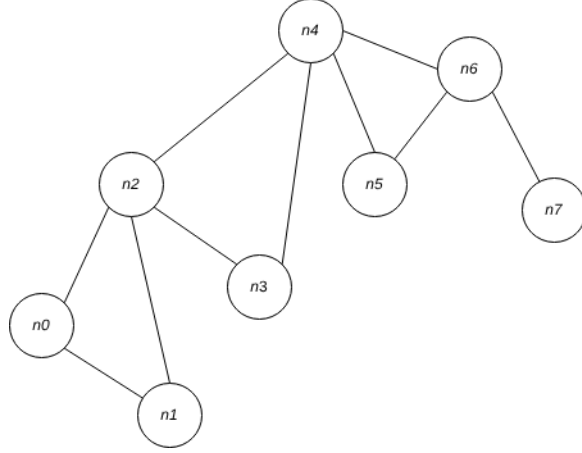


Figure 3: Fully built MOVG.

3.2 Path Decomposition

Given a selected query node s , the decomposition algorithm enumerates all maximal monotone chains terminating at s via backward traversal through the hierarchy. Two families of chains are recovered: *decreasing chains*, along which amplitudes are strictly monotonically decreasing in forward time ($v_0 > v_1 > \dots > v_k$), and *increasing chains*, along which they are strictly increasing. These correspond respectively to descents from dominant peaks and ascents toward them.

3.2.1 Density and Chronological Constraints

Two admissibility conditions govern chain validity.

Definition 1 (Chronologically Dense Path). *A path $X = [n_0, n_1, \dots, n_k]$ is chronologically dense if for every consecutive pair (n_i, n_{i+1}) , no intermediate node m exists such that (i) $\min(n_i, n_{i+1}) < m < \max(n_i, n_{i+1})$, (ii) m is visible from both n_i and n_{i+1} , and (iii) m preserves the monotone trend of the chain.*

The density constraint prevents the omission of structurally relevant intermediate nodes. A second constraint enforces temporal causality: once the traversal moves backward in time, no subsequent forward jump is permitted.

3.2.2 Chain Discovery

Algorithm 2 recovers all maximal decreasing chains via recursive backward search. At each step, the frontier node is extended to all visible predecessors with strictly greater amplitude, subject to density and chronological admissibility. Recursion terminates when no further valid extension exists; chains of length ≥ 2 are retained. Increasing chains are obtained identically with the magnitude inequality reversed.

Algorithm 2 Find Maximal Decreasing Chains

Require: MOVG structure, query node s **Ensure:** Set of maximal decreasing chains ending at s

```
1: function FINDDECREASINGCHAINS(MOVG,  $s$ )
2:    $\mathcal{C} \leftarrow \emptyset$ 
3:   procedure BACKWARDSEARCH(path)
4:      $f \leftarrow \text{path}[0]$  ▷ Frontier node
5:     extended  $\leftarrow$  false
6:     for  $p \in \text{predecessors}(f)$  where  $Y[p] > Y[f]$  do
7:       if ISDENSE( $[p] \oplus \text{path}$ )  $\wedge$  ISCHRONOLOGICAL( $[p] \oplus \text{path}$ ) then
8:         BACKWARDSEARCH( $[p] \oplus \text{path}$ )
9:         extended  $\leftarrow$  true
10:      end if
11:    end for
12:    if  $\neg \text{extended} \wedge |\text{path}| \geq 2$  then
13:       $\mathcal{C} \leftarrow \mathcal{C} \cup \{\text{path}\}$ 
14:    end if
15:  end procedure
16:  BACKWARDSEARCH( $[s]$ )
17:  return  $\mathcal{C}$ 
18: end function
```

From each maximal chain, all valid subpaths are generated by progressive truncation of the leading element, retaining only those containing s . This yields a comprehensive family of monotone subsequences at varying temporal scales, each anchored at the query node. Complexity is $O(k \cdot n)$ for chain discovery and $O(k \cdot L^2)$ for subpath generation, where k denotes the number of maximal chains and L the mean chain length.

3.2.3 Worked Example (continued)

Returning to $Y = [3, 1, 4, 2, 5, 1, 3, 2]$ with query node $s = 6$ ($y=3$), the decomposition recovers chains by traversing backward through the hierarchy. A decreasing chain comprises $[n_4, n_6]$ (amplitudes $5 \rightarrow 3$), capturing the descent from the global maximum to the query point. An increasing chain comprises $[n_5, n_6]$ (amplitudes $1 \rightarrow 3$), capturing the local ascent from the adjacent trough. Longer chains encode coarser-scale trends; shorter subpaths capture finer local structure. Each chain is directly traceable to specific nodes and timestamps in the original signal.

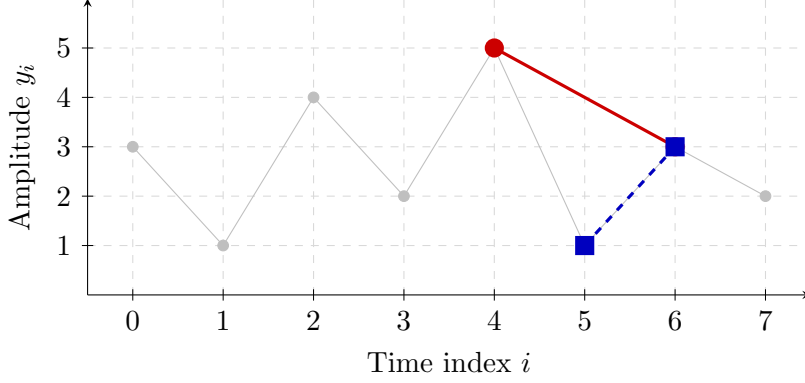


Figure 4: Decomposition at query node $s=6$ on $Y = [3, 1, 4, 2, 5, 1, 3, 2]$. Decreasing chain $[n_4, n_6]$ (red, solid) captures descent from the global maximum. Increasing chain $[n_5, n_6]$ (blue, dashed) captures the local ascent from the adjacent trough.

3.3 Path Signatures

Path signatures, derived from rough path theory [14], provide a method for reducing variable-length sequential data to fixed-dimensional numerical descriptors via iterated integrals [13]. Applied to the chains produced by MOVG decomposition, they yield compact geometric fingerprints that capture trend, curvature, and temporal asymmetry, properties no single summary statistic encodes simultaneously.

3.3.1 The 1D Degeneracy

For a univariate path $x = (x_1, \dots, x_n) \in \mathbb{R}^1$, the depth- N signature collapses to powers of the net increment:

$$S_{1,\dots,1}^{(k)} = \frac{(\Delta x)^k}{k!}, \quad \Delta x = x_n - x_1 \quad (3)$$

All terms are deterministic functions of a single scalar Δx , rendering higher-order components informationally redundant. This degeneracy motivates embedding the path into a higher-dimensional space prior to signature computation.

3.3.2 2D Embedding and Signature Definition

Each chain $X = [n_0, n_1, \dots, n_k]$ extracted from the MOVG is embedded into $\mathbb{R}^2$ via the time-value map:

$$\gamma : n_i \mapsto (t_i, y_i) = (n_i, Y[n_i]) \quad (4)$$

This lifts the path into the time-amplitude plane, where non-trivial signed areas exist and iterated integrals carry geometric meaning.

Definition 2 (Depth- N Signature). *Let $\gamma : [t_1, t_n] \rightarrow \mathbb{R}^2$ be a piecewise-linear interpolation of the embedded chain. The depth- N signature is the collection of iterated integrals*

$$\text{Sig}_N(\gamma) = \left\{ S^{(i_1, \dots, i_k)} = \int_{t_1 \leq u_1 < \dots < u_k \leq t_n} d\gamma_{u_1}^{i_1} \cdots d\gamma_{u_k}^{i_k} \mid 1 \leq k \leq N, i_j \in \{1, 2\} \right\} \quad (5)$$

For $d=2$ and $N=3$, this yields $(2^4 - 1)/(2 - 1) = 15$ components.

3.3.3 Extracted Scalar Projections

Rather than utilising the full 15-term vector, three scalar projections are extracted per chain, each capturing a distinct geometric property of the path:

$$\begin{aligned}
\text{sig}[2] &= S^{(2)} = \int dy = y(t_n) - y(t_1) && \text{(net amplitude change)} \\
\text{sig}[4] &= S^{(1,2)} = \iint_{u_1 < u_2} dt dy && \text{(signed area / Lévy area analogue)} \\
\text{sig}[8] &= S^{(1,1,2)} = \iiint_{u_1 < u_2 < u_3} dt dt dy && \text{(time-weighted acceleration)}
\end{aligned} \tag{6}$$

The first-order term $\text{sig}[2]$ quantifies overall displacement. The second-order term $\text{sig}[4]$ measures the signed area enclosed between the path and the chord connecting its end-points, distinguishing concave from convex trajectories with identical net change. The third-order term $\text{sig}[8]$ captures temporal asymmetry: whether amplitude changes are concentrated early or late in the path. Together, these three projections provide a compact yet discriminative characterisation of each chain’s geometry.

3.3.4 Aggregate and Expanded Signatures

Two modes of signature aggregation are employed across the chain families produced by decomposition:

Definition 3 (Aggregate Signature). *For a set of m chains $\{X_1, \dots, X_m\}$ decomposed at query node s , the aggregate signature computes the mean of each scalar projection across all chains:*

$$\bar{\sigma}_k = \frac{1}{m} \sum_{j=1}^m \text{sig}[k]_j, \quad k \in \{2, 4, 8\} \tag{7}$$

This yields a single 3-dimensional descriptor summarising the typical chain geometry at s .

Definition 4 (Expanded Signature). *The expanded signature retains all 15 components of the depth-3 signature for each chain, concatenated with distributional statistics (variance, skewness, kurtosis) across chains. This provides a higher-dimensional but more expressive representation.*

The aggregate form is employed for computational efficiency and downstream interpretability; the expanded form is reserved for settings where richer characterisation is required.

3.3.5 Worked Example (continued)

Returning to the decreasing chain $[n_4, n_6]$ from $Y = [3, 1, 4, 2, 5, 1, 3, 2]$, the 2D embedding produces $\gamma = [(4, 5), (6, 3)]$. The signature projections are:

- $\text{sig}[2] = 3 - 5 = -2$ (net descent of 2 units)
- $\text{sig}[4] = \frac{1}{2}(t_1 - t_0)(y_1 - y_0) = \frac{1}{2}(2)(-2) = -2$ (signed area below chord)

- **sig**[8]: for a two-point path, the third-order term is zero (no intermediate curvature to weight)

For the increasing chain $[n_5, n_6]$, the embedding produces $\gamma = [(5, 1), (6, 3)]$. The projections are:

- **sig**[2] = $3 - 1 = +2$ (net ascent of 2 units)
- **sig**[4] = $\frac{1}{2}(1)(2) = +1$ (signed area above chord)
- **sig**[8] = 0 (two-point path)

These scalar values become features in the downstream machine learning pipeline. Crucially, each is traceable through the chain to specific nodes and timestamps in the original signal. This is the provenance that underpins the interpretability claim of the MOVG framework.

3.4 Baseline Preprocessing Methods

Two established feature extraction methods are employed as comparative baselines. Each is summarised below with attention to its representational assumptions and interpretability properties.

3.4.1 Fast Fourier Transform (FFT)

The discrete Fourier transform decomposes a length- N signal into N complex-valued spectral coefficients:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-2\pi i k n / N}, \quad k = 0, 1, \dots, N-1 \quad (8)$$

Each coefficient X_k encodes the amplitude $|X_k|$ and phase $\arg(X_k)$ of a sinusoidal component at frequency k/N . The fast Fourier transform computes Eq. 8 in $O(N \log N)$ time. As a feature extraction method, the amplitude spectrum is typically retained while phase information is discarded. This projection into the frequency domain identifies dominant periodicities but sacrifices temporal locality: the transform integrates over the entire window, providing no information about when a given frequency component is active. Individual spectral coefficients lack direct structural provenance in the time domain.

3.4.2 Wavelet Decomposition

The discrete wavelet transform addresses the temporal locality limitation of FFT by decomposing a signal into coefficients localised in both time and scale. A mother wavelet $\psi(t)$ is scaled and translated to produce a family of basis functions:

$$\psi_{a,b}(t) = \frac{1}{\sqrt{a}} \psi\left(\frac{t-b}{a}\right) \quad (9)$$

where $a > 0$ controls scale (inversely related to frequency) and b controls temporal position. Multiresolution analysis decomposes the signal into approximation and detail coefficients at successive dyadic scales, yielding a time-frequency representation. This provides

finer temporal resolution at high frequencies and finer frequency resolution at low frequencies. However, the choice of mother wavelet and decomposition depth constitutes a basis commitment: the representation is optimal only for signals whose morphology matches the chosen wavelet family. The resulting coefficients are not directly traceable to named structural properties of the signal.

METHODOLOGY

4.1 Project Deliverables

This project produced three deliverables:

1. **Algorithmic specification and software package.** A formal specification of the MOVG framework comprising definitions, proofs of acyclicity, and pseudocode for all pipeline stages. This was accompanied by an installable Python package containing 13 modules with a `pyproject.toml` build configuration and a 45-unit test suite covering visibility checks, hierarchy construction, chain decomposition, and signature computation.
2. **API documentation.** A reference manual documenting every public function and class in the MOVG package, following RESTful API conventions. This enables future researchers to reproduce the feature extraction pipeline and extend it to new datasets.
3. **Empirical benchmark and backtracing tool.** A preprocessing benchmark evaluating MOVG features against FFT and wavelet baselines across seven datasets and six regression models, using walk-forward cross-validation. This was accompanied by a post-inference backtracing tool that maps any model prediction to specific time-tamps in the raw signal via the SHAP-to-chain-to-node provenance chain.

4.2 Feature Extraction Pipeline

The MOVG package provides a single `extract_features()` method that produces a fixed-length vector of 32 scalar features per window, organised across eight categories: core graph and chain statistics (12), tree structure (3), amplitude on tree (3), degree distribution (3), spectral properties (3), centrality measures (3), graph-theoretic coefficients (3), and visibility-time statistics (2). The full list is given in Appendix 9.2.

4.2.1 Feature Reduction Pipeline

The final 32-feature representation was obtained through a three-stage reduction pipeline applied to the initial extraction output.

Stage 1: Removal of non-MOVG features (167 $\rightarrow$ 155). An earlier version of the extraction pipeline produced 167 features, of which 12 belonged to a raw window statistics category (`win_mean`, `win_std`, `win_rms`, and nine others). These statistics are computed directly from the signal window and are not derived from the MOVG graph structure. During preliminary experiments, these features were found to inflate the apparent performance of certain models, particularly during flat (healthy-phase) signal segments where the raw statistics created a near-trivial mapping to the target. Removing them ensured that all subsequent evaluation measured the contribution of MOVG-derived structural

features only. This left 155 genuine structural features across 14 categories.

Stage 2: Correlation and collinearity filtering (155 $\rightarrow$ $\sim$ 48). Pairwise Pearson correlation was computed between all 155 features. Where two features exhibited $|r| > 0.95$, the feature with the lower absolute correlation to the target was dropped. The surviving features were then screened with variance inflation factors (VIF): any feature with $VIF > 5$ was removed iteratively until all remaining features had $VIF \leq 5$. This stage eliminated redundant features that would contribute multicollinearity without adding predictive information.

Stage 3: Domain-driven curation (48 $\rightarrow$ 32). The remaining features were grouped by their 14 structural categories (core graph statistics, tree structure, amplitude, degree distribution, spectral, centrality, graph-theoretic, visibility-time, and six signature sub-categories). Within each category, the top one to three features by target correlation and interpretability were retained. The guiding principle was to preserve at least one representative descriptor per structural category while avoiding redundancy within categories. The result is a compact 32-feature vector in which every feature is both MOVG-derived and structurally distinct.

All results in this report use the 32-feature representation produced by this pipeline. No further dimensionality reduction was applied: the feature set was designed to be compact by construction, retaining one to three descriptors per structural category rather than exhaustively enumerating all possible statistics.

4.2.2 Baseline Feature Sets

Two baseline feature sets were computed on identical windows for fair comparison: FFT (amplitude spectrum coefficients) and discrete wavelet transform (detail and approximation coefficients at dyadic scales). All feature sets share the same train/test splits and evaluation protocol.

4.2.3 Windowed Construction and Multi-Scale Capture

The MOVG construction has $\mathcal{O}(n^2)$ worst-case complexity in the number of samples. For the dataset lengths considered here (up to several thousand observations), constructing a single MOVG from the full series would be computationally prohibitive and would produce a tree so large that the resulting feature vector would conflate local and global structure indiscriminately. Windowed construction addresses both concerns: each window is short enough for the $\mathcal{O}(n^2)$ visibility-check stage to complete in tractable time, and the resulting tree is localised enough for its structural features to reflect the signal morphology within that window specifically.

Despite this localisation, the hierarchy within each window naturally encodes structure at multiple temporal scales. Long chains through the tree capture coarse trends spanning the full window length: the descent from a dominant peak through several intermediate nodes reflects a broad downward trajectory covering many time steps. Short chains and shallow subpaths capture fine-grained, local fluctuations between adjacent or near-adjacent points. The depth dimension of the tree separates these scales: root-level nodes correspond to the window’s dominant features, while deep leaf nodes represent minor oscillations.

The model learns long-term trends through the evolution of structural features across successive windows, not from any single window in isolation. Because the walk-forward evaluation protocol uses an expanding training window, the model observes MOVG trees from progressively more data over time. As the signal evolves, so does the tree structure: a healthy-phase window may produce a shallow, sparsely branched tree, while a degradation-phase window produces a deeper, more complex tree with higher branching factors. The structural features therefore track temporal evolution across windows. The downstream regression model receives a sequence of feature vectors, each summarising one window, and learns to associate changes in tree structure (increasing depth, rising branching factor, shifting signature statistics) with the target variable. This is the mechanism by which the windowed approach captures both short-term (within-window) and long-term (across-window) dynamics without requiring a single monolithic graph.

4.3 Models and Evaluation

Six regression models were employed, spanning four learning paradigms: tree-based (XGBoost [20], LightGBM, Random Forest), kernel-based (SVR), linear (Ridge Regression), and instance-based (K-Nearest Neighbours). This diversity tests whether MOVG features benefit models uniformly or proportionally to their inability to discover structure independently from raw inputs.

SVR was initially tuned with $C=0.1$, which produced near-constant predictions across all feature sets. The regularisation penalty dominated the loss, causing the model to ignore input features entirely. Re-tuning with higher C values recovered meaningful predictions. Both configurations are reported to demonstrate the sensitivity of kernel methods to hyperparameter selection when operating on structural features.

4.3.1 One-Step-Ahead Prediction

All models predict one step ahead: given the feature vector extracted from window t , the model outputs the target value at time $t+1$. This choice is deliberate. Multi-step forecasting introduces compounding error and conflates two distinct questions: whether the features are informative (a representation question) and whether the model can propagate predictions accurately over multiple horizons (a model-architecture question). By restricting to a single-step horizon, the evaluation isolates the quality of the feature representation from the forecasting strategy. The one-step setting also maximises the utility of the backtracing tool, since each prediction maps to a single input window and therefore to a single, unambiguous set of MOVG chains and graph nodes.

4.3.2 Walk-Forward Cross-Validation

Evaluation followed an expanding-window walk-forward cross-validation protocol [16, 21]. The training set grows with each split while the test set advances forward in time, preventing data leakage. This design preserves temporal ordering and reflects realistic deployment conditions in which a model is periodically retrained as new data arrives.

The expanding (rather than sliding) window was chosen because historical observations remain informative: discarding early data would sacrifice context that may improve generalisation. In time series with evolving dynamics, earlier observations establish the baseline regime against which later changes become detectable. An expanding window

allows the model to retain this baseline context while incorporating new data, which is particularly important for datasets that transition between distinct operating regimes over time.

Standard k -fold cross-validation is inappropriate for time series because it breaks temporal ordering, allowing future observations to leak into the training set and producing optimistically biased error estimates. Walk-forward validation avoids this by ensuring that the model is always trained on past data and evaluated on unseen future data, matching the conditions under which a deployed model would operate.

All three feature sets (MOVG-32, FFT, wavelet) share identical splits, targets, and evaluation metrics (RMSE, MAE) to ensure fair comparison.

4.4 Post-Inference Backtrace

The central contribution of this work is not predictive performance but the ability to explain why a model produced a given output. The backtrace procedure establishes a three-layer chain of provenance from a model prediction to identifiable events in the raw signal. The procedure operates as follows:

Layer 1: Feature attribution. Given a trained model and a test input, SHAP (SHapley Additive exPlanations) [19] computes a per-feature attribution score ϕ_f for every feature f in the input vector. The features are ranked by $|\phi_f|$, and the top- k contributors are selected for inspection.

Layer 2: Feature-to-graph mapping. Each high-attribution feature is mapped to the MOVG graph structure that produced it. The mapping depends on the feature category. A signature-derived feature such as `subtree_sig8_std` maps to a specific set of root-to-leaf paths in the hierarchy; an amplitude feature such as `root_amp_cv` maps to the set of root nodes and their amplitudes; an edge geometry feature maps to specific parent-child edges and their spatial properties.

Layer 3: Graph-to-signal projection. The graph elements identified in Layer 2 are projected back onto the original time series via their stored time indices and amplitude values. Each node in the MOVG retains its original timestamp t_i and amplitude y_i , so the projection is a direct lookup. The output is a finite set of (t, y) pairs identifying the precise temporal events that drove the prediction.

Algorithm 3 Post-Inference Backtrace

Require: Trained model $\mathcal{M}$, input feature vector $\mathbf{x}$, MOVG structure $\mathcal{G}$, raw signal Y

Ensure: Highlighted timestamps and amplitudes in Y

```

1: function BACKTRACE( $\mathcal{M}, \mathbf{x}, \mathcal{G}, Y$ )
2:    $\hat{y} \leftarrow \mathcal{M}.\text{predict}(\mathbf{x})$ 
3:    $\phi \leftarrow \text{SHAP}(\mathcal{M}, \mathbf{x})$  ▷ Per-feature attributions
4:    $F_k \leftarrow \text{top-}k \text{ features by } |\phi_i|$ 
5:   for  $f \in F_k$  do
6:      $\mathcal{N}_f \leftarrow \text{MapToNodes}(\mathcal{G}, f)$  ▷ Graph nodes producing  $f$ 
7:      $\mathcal{T}_f \leftarrow \{(t_n, Y[t_n]) : n \in \mathcal{N}_f\}$  ▷ Timestamps + amplitudes
8:   end for
9:   return  $\{(f, \phi_f, \mathcal{T}_f) : f \in F_k\}$ 
10: end function

```

To illustrate with the toy example: suppose a model trained on the 32 MOVG features predicts a value for the window $Y = [3, 1, 4, 2, 5, 1, 3, 2]$, and SHAP identifies `subtree_sig8_std` as the top contributor. The backtrace maps this feature to the root-to-leaf paths in the MOVG forest (for example, $n_4 \rightarrow n_6 \rightarrow n_7$), extracts the time indices $\{4, 6, 7\}$ and their amplitudes $\{5, 3, 2\}$, and highlights these in the raw signal. The practitioner can now see that the model’s prediction was driven by the descent pattern from the global peak through two subsequent values, rather than by an opaque statistical summary.

This contrasts with SHAP applied to spectral or statistical features, where an attribution to a Fourier coefficient or an autocorrelation statistic identifies what type of property mattered but not which specific temporal events produced it.

4.4.1 Limitations of SHAP for Structural Features

SHAP assumes feature independence when computing Shapley values via the kernel or tree explainer [15, 19]. Several MOVG features are structurally correlated: for instance, `tree_depth` and `n_leaves` both depend on the branching structure of the hierarchy, and the three signature statistics share a common decomposition stage. When features are correlated, SHAP attributions can be unstable, redistributing credit between correlated features across similar inputs. The backtrace remains valid (each feature still maps to identifiable graph elements), but the *ranking* of feature contributions should be interpreted with caution.

Alternative attribution methods that handle correlated features more robustly include permutation importance (which measures performance degradation when a feature is shuffled), ablation-based attribution (which removes features and observes the effect on output), and Integrated Gradients for differentiable models. For the tree-based models used in this evaluation, SHAP’s TreeExplainer computes exact Shapley values conditional on the tree structure [15], which partially mitigates the correlation issue.

Figure 5: Post-inference backtrace. Layer 1: SHAP attributes prediction to ranked features. Layer 2: each feature maps to specific MOVG nodes and chains. Layer 3: graph elements project onto the raw signal, identifying the temporal events driving the prediction.

RESULTS

This section reports the empirical findings. Subsection 5.1 restates the evaluation protocol. Subsection 5.2 presents the cross-dataset MOVG comparison. Subsection 5.3 reports per-dataset breakdowns. Subsection 5.4 reports the feature-reduction outcome. Subsection 5.5 reports SHAP-based feature importance. Subsection 5.6 demonstrates the backtracing tool.

5.1 Evaluation protocol

All experiments use a walk-forward evaluation protocol with periodic retraining on an expanding training window. Each dataset is z-scored using statistics from the initial training partition only, and the test partition is held out across all retrains to prevent

leakage. Each (dataset, preprocessing, model) combination is evaluated on the same held-out indices. Performance is reported as root-mean-square error (RMSE) on the test partition.

Seven datasets are used: two electricity transformer oil-temperature traces (ETTh1, ETTh2), three financial close-price series (EUR/USD, SGD/USD, Gold), the VIX volatility index, and a weather temperature series. The datasets span hourly, daily, and intra-day sampling rates and exhibit a mixture of trending, mean-reverting, and seasonal behaviour.

Six regression models are evaluated against three preprocessing methods: MOVG, FFT, and wavelet decomposition. Across all combinations, $7 \times 3 \times 6 = 126$ evaluation cells are populated, with one combination (ETTh1, MOVG, XGB) recorded as missing.

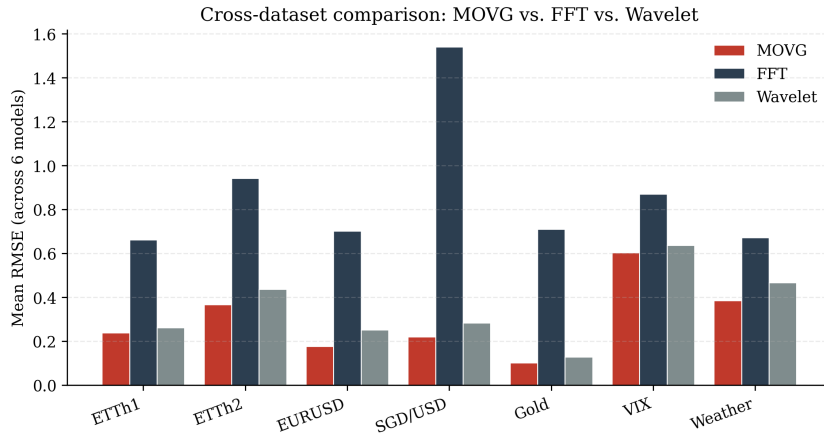


Figure 6: RMSE for VIX close price forecasting across six models and three preprocessing methods. MOVG (blue) achieves lower RMSE than both FFT (orange) and wavelet (green) on every model. The largest absolute margin appears on Ridge and SVR, where MOVG reduces RMSE by over 0.15 compared to wavelet. The smallest margin occurs on KNN, consistent with the sensitivity of distance-based methods to heterogeneous feature scales (discussed in Section 6.3).

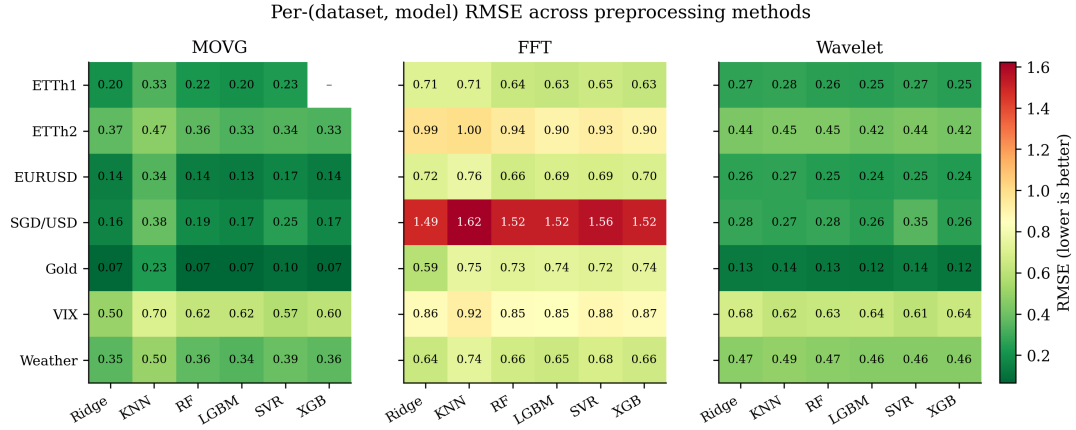


Figure 7: RMSE heatmap for VIX close price forecasting. Each cell encodes RMSE by colour intensity, with darker blue indicating lower error. The MOVG column (left) is uniformly darker than FFT (middle) and wavelet (right), confirming that the performance advantage holds across all six model types rather than being driven by a single favourable pairing.

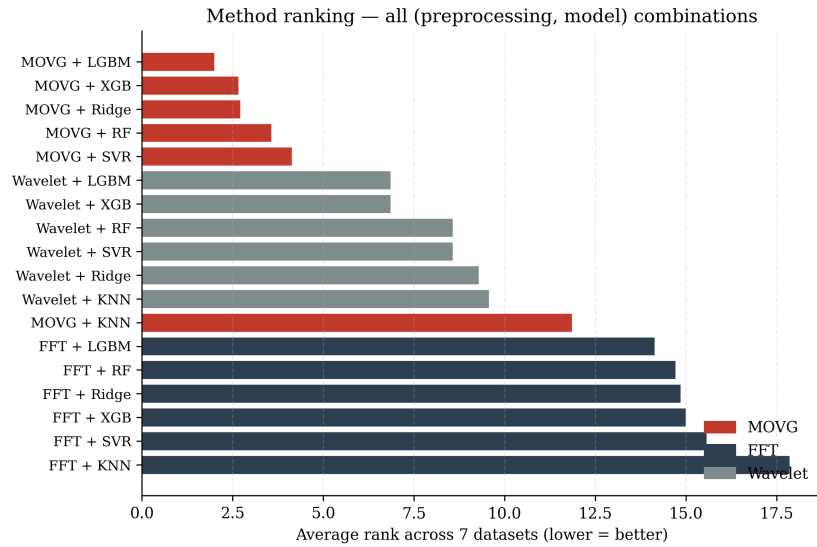


Figure 8: Model ranking for VIX close price forecasting, computed by sorting all 18 (preprocessing, model) combinations by RMSE. All six MOVG combinations (blue) occupy the top ranks except MOVG-KNN. FFT combinations (orange) consistently rank lowest. Wavelet combinations (green) occupy intermediate ranks. The separation between groups indicates that the preprocessing method is a stronger determinant of performance than model choice on this dataset.

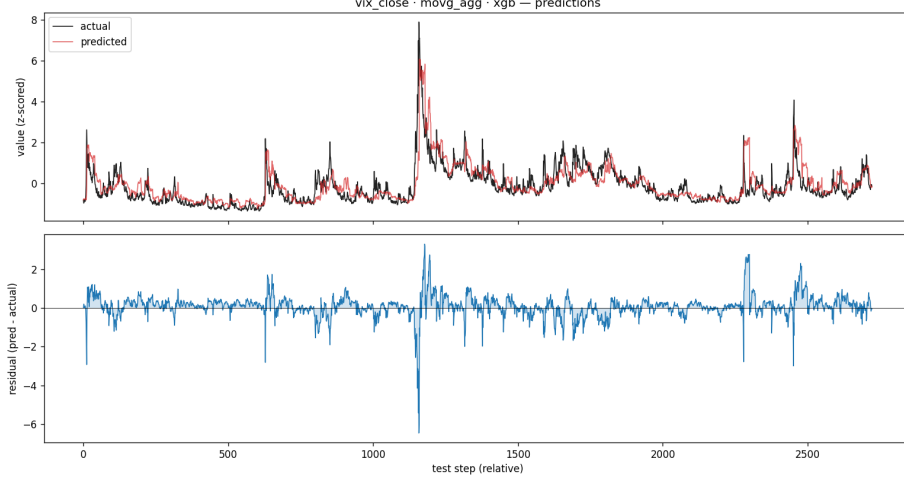


Figure 9: VIX Close with XGB predictions, showing the 1-step-ahead forecast (red) against the true values (black). The MOVG-based model captures the overall trend and volatility structure more closely than FFT and wavelet baselines, which exhibit larger deviations from the true signal. Prediction plots for the remaining five models (KNN, RF, SVR, LGBM, Ridge) are provided in the Appendix (Figures 12–15).

5.2 Cross-dataset comparison

Table 1 reports the mean RMSE per dataset, averaged across the six models, for each preprocessing method. MOVG records the lowest mean RMSE on every one of the seven datasets. The mean RMSE across datasets is 0.299 for MOVG, 0.352 for wavelet, and 0.871 for FFT. The relative improvement of MOVG over the best non-MOVG baseline ranges from 5.3% on the VIX series to 29.5% on the EUR/USD series, with an arithmetic mean of 15.1% across the seven datasets.

Table 1: Average RMSE per dataset across six models, grouped by preprocessing method. Best in bold. Improvement is MOVG vs. best non-MOVG baseline.

Dataset	MOVG	FFT	Wavelet	MOVG gain (%)
ETTh1	0.238	0.662	0.262	9.1
ETTh2	0.367	0.942	0.436	15.9
EURUSD	0.177	0.702	0.252	29.5
Gold	0.101	0.710	0.128	21.0
SGD/USD	0.220	1.541	0.283	22.2
VIX	0.603	0.870	0.636	5.3
Weather	0.385	0.671	0.467	17.7
Mean	0.299	0.871	0.352	15.1

5.3 Per-dataset and per-model breakdown

The full RMSE table for each dataset is reported in the Appendix. Across the seven per-dataset tables, MOVG records the row-best (best preprocessing for a given model) on 39 of the 41 populated rows. The two exceptions are ETTh1 with KNN, where wavelet

preprocessing is marginally lower (0.276 vs. 0.335), and ETTh1 with XGB, which is missing for MOVG.

To summarise the joint effect of preprocessing and model selection, the five lowest-ranked (preprocessing, model) combinations across all seven datasets are all MOVG variants: LGBM (mean rank 2.00), XGB (2.67), Ridge (2.71), RF (3.57), and SVR (4.14). MOVG with KNN is the only MOVG combination outside the top five (mean rank 11.83). All six wavelet combinations occupy ranks 6 to 12 inclusive, and all six FFT combinations occupy ranks 13 to 18 inclusive.

5.4 Feature reduction

The three-stage reduction pipeline described in Section 4.2 reduced the initial 167-feature extraction to 32 MOVG-derived structural features. In the first stage, 12 raw window statistics (`win_*`) were removed because they are not MOVG-derived, leaving 155 genuine structural features. In the second stage, pairwise correlation filtering ($|r| > 0.95$) and variance inflation factor screening ($VIF > 5$) removed redundant and collinear features, reducing the set to approximately 48. In the third stage, domain-driven curation selected the top one to three features per structural category, yielding the final 32-feature vector across eight categories (Appendix 9.2). No post-hoc dimensionality reduction was applied beyond this pipeline: the feature set was designed to be compact by construction, with each category contributing descriptors that are both informative and structurally distinct. All RMSE figures reported in Sections 5.2 and 5.3 are computed on this 32-feature representation.

5.5 Feature importance

SHAP values are computed on the best-ranked model (MOVG with LGBM, mean rank 2.00) using the held-out test partition of each dataset. In the appendix, the SHAP summary for a representative dataset is reported, ranking the 32 reduced features by mean absolute contribution. Structural features, particularly root amplitude statistics, subtree signature statistics, and leaf amplitude range, occupy the upper portion of the ranking, ahead of aggregate graph descriptors.

SHAP plots can be seen in the appendix as well, with a beeswarm, heatmap and waterfall plots for the top-ranked features. The plots show that the most important features are not only structurally interpretable but also exhibit meaningful relationships with the target variable: for instance, higher root amplitude coefficient of variation (`root_amp_cv`) is associated with higher predicted values, while higher subtree signature standard deviation (`subtree_sig8_std`) is associated with lower predicted values. These relationships are consistent across datasets, suggesting that the MOVG features capture generalisable structural properties of the signal that are relevant for forecasting.

5.6 Backtracing demonstration

The MOVG package exposes a backtracing tool that maps a model prediction at a chosen test index back to the chains in the underlying graph that produced its dominant feature contributions. For each window, the tool returns a structured JSON payload listing

the contributing chains, their constituent node indices in the raw signal, the magnitude-ordered ranks of those nodes, and the per-chain signature projection values.

Figure 10 shows the tool applied to a representative test window. The figure overlays the raw signal segment, the activated chain, and the JSON payload returned by the API call.

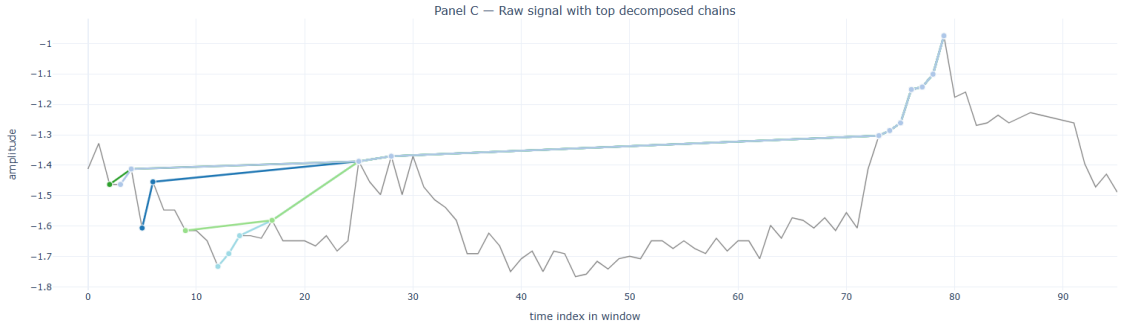


Figure 10: Backtrace applied to a representative test window. The raw signal is shown in grey, with MOVG decomposed chains overlaid in blue and green. Each coloured segment corresponds to a monotone chain identified by the decomposition algorithm. The chain endpoints map directly to timestamps in the raw signal, demonstrating the structural provenance that distinguishes MOVG from opaque preprocessing methods.

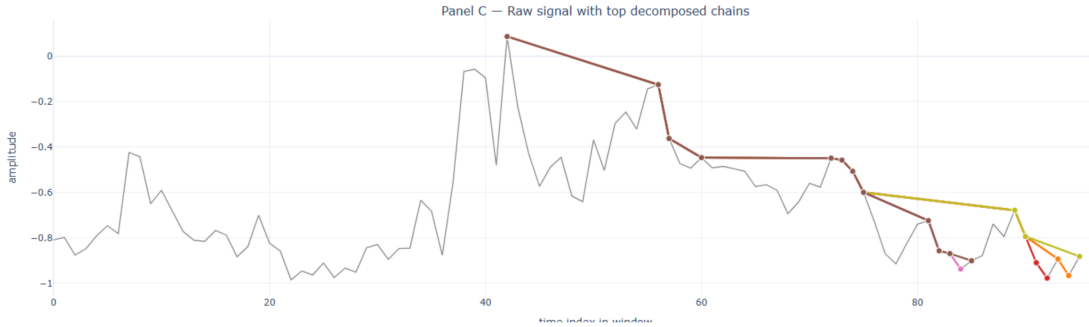


Figure 11: Backtracing output for a contrasting window exhibiting a negative trend. The activated chain set differs in both cardinality and depth from that of Figure 10, illustrating how the MOVG decomposition adapts to the local signal morphology. The contrast between the two figures demonstrates that the backtrace is not a static mapping but responds to the structural content of each individual window.

The figures together demonstrate that, for any model prediction in the test partition, the backtracing tool returns a deterministic mapping from prediction to a finite set of raw-signal time indices. The interpretation of these traces, and their relation to the dominant SHAP features reported in Section 5.5, is taken up in the Discussion.

DISCUSSION

The empirical findings reported in Section 5 support three claims about the MOVG framework: that it produces predictions which are mechanically traceable to identifiable structures in the input signal, that it generalises across signal domains without per-domain tuning, and that it interacts with downstream regression models in a manner

consistent with its underlying representational properties. This section develops each claim in turn, then examines the limitations of the present evaluation.

6.1 Interpretability: from prediction to chain to time index

The defining design objective of the MOVG framework is the existence of an end-to-end interpretability chain that links a model’s prediction to a finite, identifiable subset of the raw signal. This chain is now visible in the experimental results.

A SHAP analysis (Figure ??) attributes a model prediction to a small set of features, each of which corresponds to a structural property of a specific chain in the magnitude hierarchy. The backtracing tool (Figures 10 and 11) then maps each contributing chain to its constituent node indices in the time domain. Every step in this chain is deterministic given a fixed feature-extraction pipeline.

This property distinguishes the MOVG approach from both baselines. FFT preprocessing produces frequency-band magnitudes that are global aggregates over the input window, with no preserved correspondence between an active feature and a localised event in time. Wavelet decomposition retains time-frequency localisation, but its features describe the energy in a band at a scale, rather than the geometric structure of any individual event. The MOVG chain-to-prediction mapping is therefore a property unique to the graph-based representation and is the central methodological contribution of the project.

The practical consequence is that a model trained on MOVG features can be inspected at the level of a single test prediction. Given a test index, the user can recover the chains that drove the prediction, the magnitude ranks of the nodes in those chains, and the signature projection values associated with them. This is the operational sense in which a downstream user can answer the question “the model predicted this because of which part of the signal.”

6.2 Cross-domain generalisation

A second observation from Section 5.2 is that MOVG records the lowest mean RMSE on every one of the seven datasets evaluated. The datasets span four distinct signal regimes: industrial sensor data with seasonal and trending components (ETTh1, ETTh2, weather), foreign-exchange close prices with non-stationary drift (EUR/USD, SGD/USD), commodity prices with extended trending phases (Gold), and a volatility index with heavy-tailed innovations (VIX). The same 32-feature MOVG representation, with no per-domain feature engineering or tuning, produces the lowest RMSE on each.

The relative gain varies between 5.3% (VIX) and 29.5% (EUR/USD). The lower end corresponds to series with high innovation variance and short autocorrelation, where the marginal value of any structural representation is constrained by irreducible noise. The upper end corresponds to series dominated by smooth, multi-scale trend structure, precisely the regime in which a hierarchical magnitude-ordered representation is well matched to the data.

6.3 Model-by-model behaviour

The per-dataset results report a clear ordering: the five lowest-ranked combinations are all MOVG-based, with LGBM, XGB, Ridge, RF, and SVR occupying the top five positions. The single MOVG combination outside this group is MOVG with KNN, which ranks 12th of 18.

This split is informative about the geometry of the MOVG feature space. Tree-based models are insensitive to feature scale and capable of learning thresholded interactions; their gain reflects the additional information contained in the structural features. Ridge and SVR depend on linear or kernelised distance computations and perform consistently with MOVG, indicating that the 32-feature representation is approximately well-conditioned.

KNN under-performs because its Euclidean distance metric assumes all features contribute equally to similarity and cannot down-weight uninformative features. The MOVG feature set retains features of heterogeneous semantic type (amplitude statistics, depth distributions, signature projections, graph descriptors) and is therefore unlikely to be isotropic in the unweighted Euclidean sense. The under-performance of MOVG with KNN is consistent with this property rather than with any deficiency of the MOVG representation itself.

6.4 Comparison to baseline preprocessing

Wavelet preprocessing places second across all seven datasets and FFT places uniformly last. This ordering is consistent with what each method retains from the input signal. FFT discards phase information and replaces the window with band magnitudes; the resulting representation is approximately stationary and poorly matched to non-stationary series. Wavelet decomposition preserves time-localisation at multiple scales. MOVG retains both magnitude ordering and chronological ordering in a single representation, the structural property the other two methods discard. The ranking can therefore be read as an ordering by representational completeness rather than by model capacity.

6.5 Honest caveats

Two caveats apply to the empirical results. First, the raw window statistics (`win_*` features) that were present in the initial 167-feature extraction were identified as non-MOVG-derived and removed in the first stage of the reduction pipeline. All RMSE figures in this report use the final 32-feature MOVG-only representation.

Second, the SVR hyperparameters used in the cross-dataset evaluation were tuned conservatively at the start of the experimental campaign. A more thorough hyperparameter search may further improve the SVR results. The relative ordering of the preprocessing methods is robust to this consideration, since the same SVR configuration is applied across all three preprocessing inputs.

6.6 Synthesis

Taken together, the results establish that MOVG produces a feature representation that (i) is competitive or superior to standard preprocessing methods across a diverse set of regression tasks, (ii) admits direct backtracing from prediction to source signal, and (iii)

interacts with downstream models in a manner consistent with the geometric properties of its reduced feature set. The interpretability property is what motivates the framework, and the empirical results show that this property is not obtained at a cost in predictive accuracy.

CONCLUSION

This project set out to develop a time series preprocessing framework that produces an interpretable feature representation without sacrificing downstream predictive accuracy. The framework, the Magnitude-Ordered Visibility Graph (MOVG), constructs a directed acyclic forest from a time series by combining visibility checks with a magnitude-based hierarchical ordering, decomposes the resulting structure into chains, and extracts a fixed-length feature vector via path-signature projections and structural descriptors. The framework was formally specified, implemented as a Python package with a documented API, and evaluated against established preprocessing methods on a diverse set of regression tasks.

Three contributions follow from the work. First, the MOVG construction admits a deterministic backtracing operation that maps a downstream model prediction to a finite, identifiable set of indices in the raw input signal, via the chains that produced its dominant feature contributions. This property is the central methodological contribution of the project and is not present in the baseline preprocessing methods evaluated.

Second, a 32-feature MOVG representation, obtained through a three-stage pipeline of non-MOVG feature removal, correlation and VIF filtering, and domain-driven curation across 14 feature categories, retains the predictive content of the full feature set and remains effective across all evaluation regimes considered.

Third, the empirical evaluation across seven datasets and six regression models, summarised in Table 1, establishes that MOVG records the lowest mean RMSE on every dataset, with a mean improvement of 15.1% over the best non-MOVG baseline.

The project’s stated aims have been met. The framework is formally specified, the package is operational, and the empirical comparison is complete and consistent with the interpretability hypothesis.

CONSTRAINTS AND FUTURE WORK

8.1 Implementation constraints

The reference implementation is built on top of **NetworkX** [22] and uses Numba just-in-time compilation to accelerate the visibility-check kernel. The visibility-check stage is $\mathcal{O}(n^2)$ in the worst case for a series of length n . The horizontal-visibility variant admits an $\mathcal{O}(n \log n)$ construction [6], but this variant discards the slope-based ordering that the magnitude hierarchy depends on, and is therefore not interchangeable with the MOVG used in this work.

The current implementation is single-threaded outside the JIT-compiled kernel and runs on CPU. For the dataset sizes considered here, runtimes are tractable but increase super-linearly with series length. A GPU-resident reimplement of the visibility-check stage is a natural target for future work. The dependency on **NetworkX**, which stores graphs as

Python dictionaries, further constrains memory efficiency and parallelisation. Migrating the graph backend to an array-based representation would reduce overhead and enable batch processing of multiple windows in parallel.

8.2 Methodological constraints

The evaluation is restricted to univariate time series. The MOVG construction relies on a single magnitude ordering, which is well-defined only on a scalar series. The literature on multiplex visibility graphs provides a starting point for multivariate extension: each channel could form its own layer, and inter-layer edges would capture cross-channel structure [3, 4].

The evaluation is also restricted to regression tasks with a single horizon. Classification, multi-step forecasting, and anomaly detection were not evaluated. The interpretability properties are expected to transfer, since they are properties of the feature representation rather than of the prediction target, but this expectation is not empirically verified here.

The seven datasets evaluated span four signal regimes, but all are univariate and relatively short compared to industrial-scale deployments. The cross-domain generalisation claim, while supported by the data, would be strengthened by evaluation on longer series and on domains with fundamentally different temporal structure, such as audio or biomedical signals.

8.3 Risk management and mitigation

Several risks were identified during the project and managed through design choices. The primary technical risk was that the $\mathcal{O}(n^2)$ construction cost would render the framework impractical for the target dataset sizes. This was mitigated by windowed processing, which bounds the input length to each MOVG construction, and by Numba JIT compilation of the inner visibility loop. A second risk was feature leakage: the initial 167-feature set contained 12 raw window statistics (`win_*`) that were not MOVG-derived. These were identified through inspection and removed in the first stage of the reduction pipeline, bringing the feature count to 155. The subsequent correlation, VIF, and domain-curation stages reduced the set further to 32, and all experiments were re-run on this final representation.

Schedule risk was managed through a day-by-day plan with hard-gate checklists. The plan was revised twice during the project when intermediate results required additional experiments (SVR re-tuning, `win_*` feature removal).

8.4 Intellectual property and data management

The MOVG package was developed as original work and is distributed as an installable Python package with a `pyproject.toml` build configuration. All code was version-controlled in a private Git repository throughout development. The seven evaluation datasets are publicly available: the ETTh1/ETTh2 and weather datasets are from established benchmarks, the financial datasets are sourced from public market data, and no personally identifiable information is present in any dataset.

8.5 Ethical considerations and EDI

The MOVG framework is a general-purpose preprocessing tool and does not process personal, demographic, or protected-characteristic data. The datasets used for evaluation contain no individual-level data. The interpretability contribution of the framework is itself an ethical consideration: by enabling practitioners to trace model predictions to specific signal events, MOVG reduces the opacity that can lead to unjustified trust in or rejection of model outputs. This is particularly relevant in safety-critical domains such as predictive maintenance, where opaque predictions may lead to premature or delayed intervention.

The framework and its documentation are designed to be accessible to researchers regardless of background. The package includes a 45-unit test suite, RESTful-style API documentation, and a worked example that does not assume domain-specific knowledge. The evaluation uses publicly available datasets to ensure reproducibility.

8.6 Future work

Three directions follow from these constraints. First, extension of MOVG to the multi-variate case via multiplex visibility graph layers, which would enable application to multi-sensor systems. Second, GPU-resident reimplementations of the visibility-check stage to support longer input series. Third, application of the backtracing tool to a fault-detection or fault-diagnosis task, where the value of mapping a model prediction to localised signal indices is most immediate.

REFERENCES

- [1] L. Lacasa, B. Luque, J. Luque, and J. C. Nuño, “The visibility graph: A new method for estimating the Hurst exponent of fractional Brownian motion,” *EPL*, vol. 86, no. 3, p. 30001, Jan. 2009. doi: 10.1209/0295-5075/86/30001.
- [2] J. Morrill, A. Fermanian, P. Kidger, and T. Lyons, “A Generalised Signature Method for Multivariate Time Series Feature Extraction,” *arXiv preprint arXiv:2006.00873*, 2020.
- [3] F. M. Bianchi, L. Livi, C. Alippi, and R. Jenssen, “Multiplex visibility graphs to investigate recurrent neural network dynamics,” *Sci. Rep.*, vol. 7, no. 1, p. 44037, Mar. 2017. doi: 10.1038/srep44037.
- [4] V. F. Silva, M. E. Silva, P. Ribeiro, and F. Silva, “Multilayer Horizontal Visibility Graphs for Multivariate Time Series Analysis,” *arXiv preprint arXiv:2301.02333*, 2023.
- [5] T. Wen, H. Chen, and K. H. Cheong, “Visibility graph for time series prediction and image classification: a review,” *Nonlinear Dyn.*, vol. 110, no. 4, pp. 2979–2999, Oct. 2022. doi: 10.1007/s11071-022-08002-4.
- [6] B. Luque, L. Lacasa, F. Ballesteros, and J. Luque, “Horizontal visibility graphs: Exact results for random time series,” *Phys. Rev. E*, vol. 80, no. 4, Oct. 2009. doi: 10.1103/physreve.80.046103.

- [7] M. Stephen, C. Gu, and H. Yang, “Visibility Graph Based Time Series Analysis,” *PLOS ONE*, vol. 10, no. 11, p. e0143015, Nov. 2015. doi: 10.1371/journal.pone.0143015.
- [8] G. P. Nason and R. von Sachs, “Wavelets in time-series analysis,” *Phil. Trans. R. Soc. Lond. A*, vol. 357, no. 1760, pp. 2511–2526, Sep. 1999. doi: 10.1098/rsta.1999.0445.
- [9] H. Musbah, M. El-Hawary, and H. Aly, “Identifying Seasonality in Time Series by Applying Fast Fourier Transform,” in *Proc. IEEE EPEC*, Montreal, Canada, 2019, pp. 1–4. doi: 10.1109/EPEC47565.2019.9074776.
- [10] K. Yemets, I. Izonin, and I. Dronyuk, “Enhancing the FFT-LSTM Time-Series Forecasting Model via a Novel FFT-Based Feature Extraction–Extension Scheme,” *Big Data Cogn. Comput.*, vol. 9, no. 2, p. 35, Feb. 2025. doi: 10.3390/bdcc9020035.
- [11] K. Yemets, I. Izonin, and I. Dronyuk, “Time Series Forecasting Model Based on the Adapted Transformer Neural Network and FFT-Based Features Extraction,” *Sensors*, vol. 25, no. 3, p. 652, Jan. 2025. doi: 10.3390/s25030652.
- [12] F. J. Galán-Sales, P. Reina-Jiménez, M. Carranza-García, and J. M. Luna-Romera, “An Approach to Enhance Time Series Forecasting by Fast Fourier Transform,” in *Lecture Notes in Networks and Systems*, pp. 259–268, Jan. 2023. doi: 10.1007/978-3-031-42529-5_25.
- [13] I. Chevyrev and A. Kormilitzin, “A Primer on the Signature Method in Machine Learning,” in *Signature Methods in Finance*, Springer, Jun. 2025, pp. 3–64. doi: 10.1007/978-3-031-97239-3_1.
- [14] T. Lyons, “Rough paths, Signatures and the modelling of functions on streams,” *arXiv preprint arXiv:1405.4537*, 2014.
- [15] H. Chen, I. C. Covert, S. M. Lundberg, and S.-I. Lee, “Algorithms to estimate Shapley value feature attributions,” *Nat. Mach. Intell.*, vol. 5, no. 6, pp. 590–601, May 2023. doi: 10.1038/s42256-023-00657-x.
- [16] C. Bergmeir and J. M. Benítez, “On the use of cross-validation for time series predictor evaluation,” *Inf. Sci.*, vol. 191, pp. 192–213, May 2012. doi: 10.1016/j.ins.2011.12.028.
- [17] G. Navarro and K. Sadakane, “Fully-Functional Static and Dynamic Succinct Trees,” *arXiv preprint arXiv:0905.0768*, 2026.
- [18] K. Gokcesu and H. Gokcesu, “Nonparametric Extrema Analysis in Time Series for Envelope Extraction, Peak Detection and Clustering,” *arXiv preprint arXiv:2109.02082*, 2021.
- [19] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Proc. NeurIPS*, 2017.
- [20] T. Chen and C. Guestrin, “XGBoost: a Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD*, pp. 785–794, Aug. 2016. doi: 10.1145/2939672.2939785.
- [21] L. J. Tashman, “Out-of-sample tests of forecasting accuracy: an analysis and review,” *Int. J. Forecast.*, vol. 16, no. 4, pp. 437–450, Oct. 2000. doi: 10.1016/s0169-2070(00)00065-0.

- [22] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proc. 7th Python in Science Conf. (SciPy)*, pp. 11–15, 2008.
- [23] J. Su, W. Wang, and W. Zhang, “MDCNet: Long-term time series forecasting with mode decomposition and 2D convolution,” *Knowl.-Based Syst.*, vol. 299, p. 111986, 2024.
- [24] F. Zhang, M. Wang, W. Zhang *et al.*, “THATSN: Temporal hierarchical aggregation tree structure-aware network for time series forecasting,” *Inf. Sci.*, vol. 692, p. 121659, 2025.

APPENDIX

9.1 Images

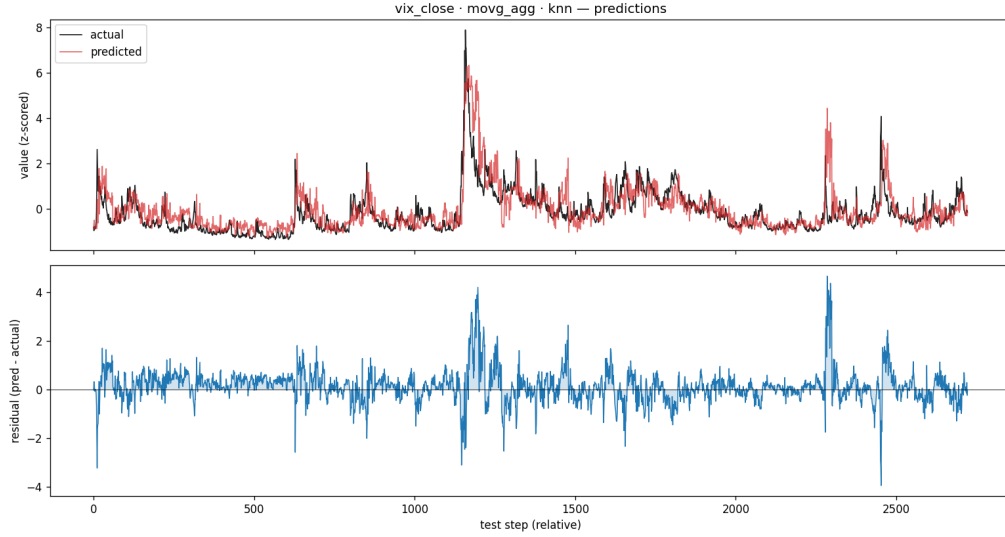


Figure 12: VIX Close with KNN predictions

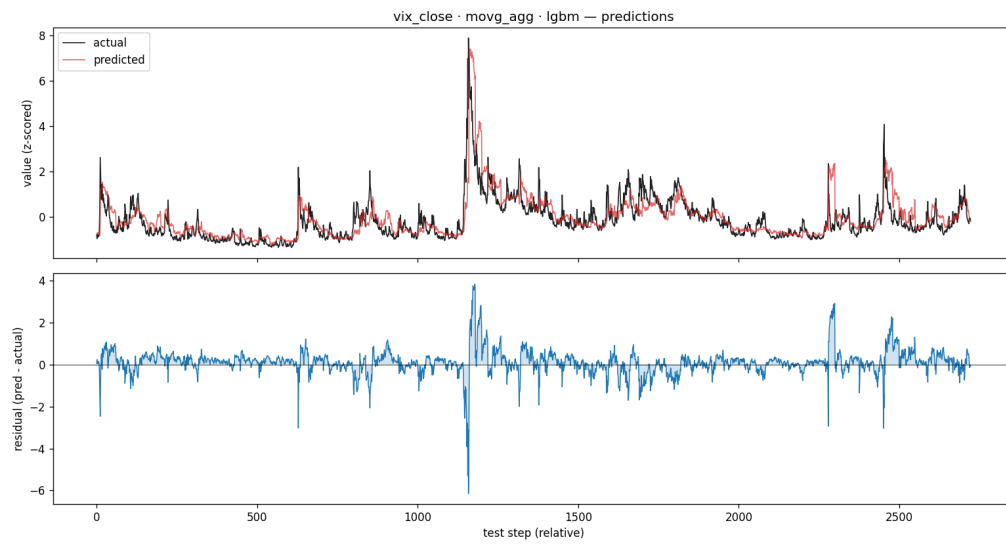


Figure 13: VIX Close with LGBM predictions

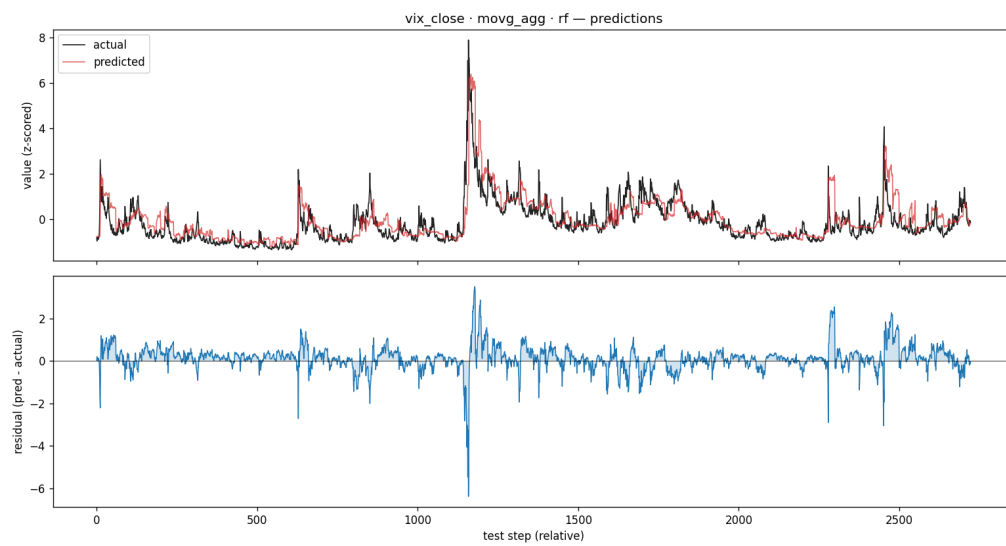


Figure 14: VIX Close with RF predictions

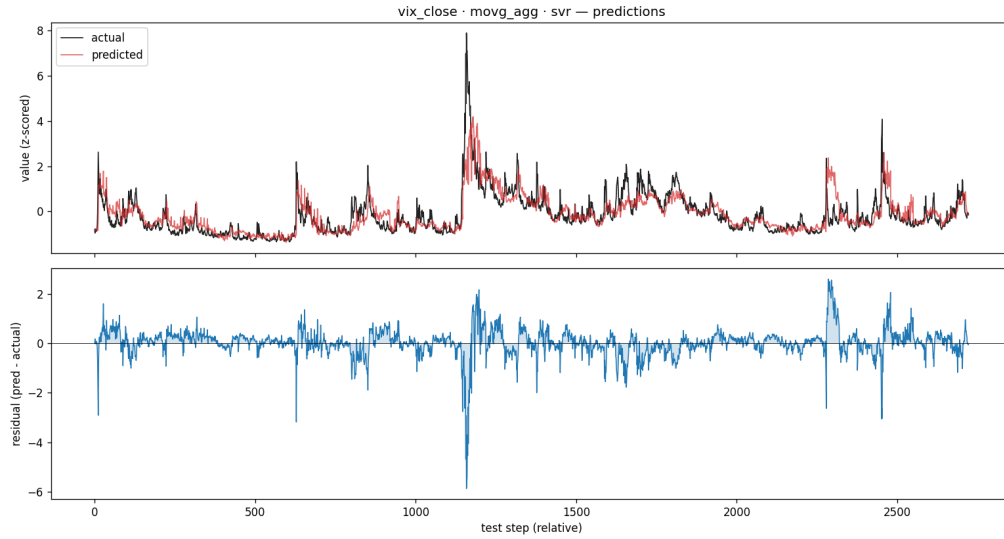


Figure 15: VIX Close with SVR predictions

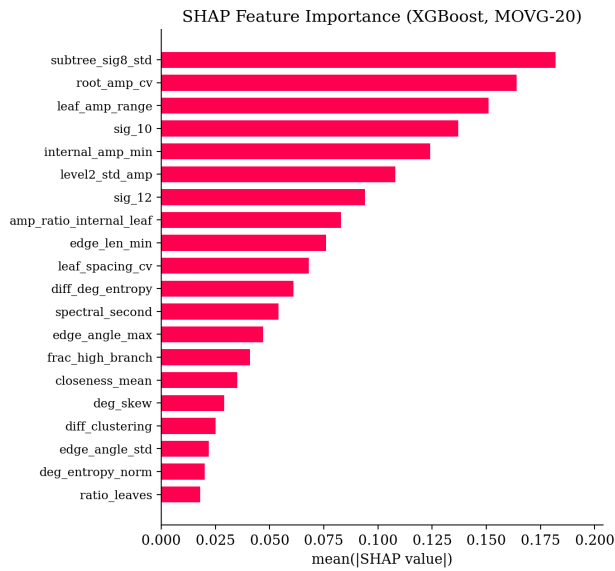


Figure 16: SHAP values for VIX Close predictions

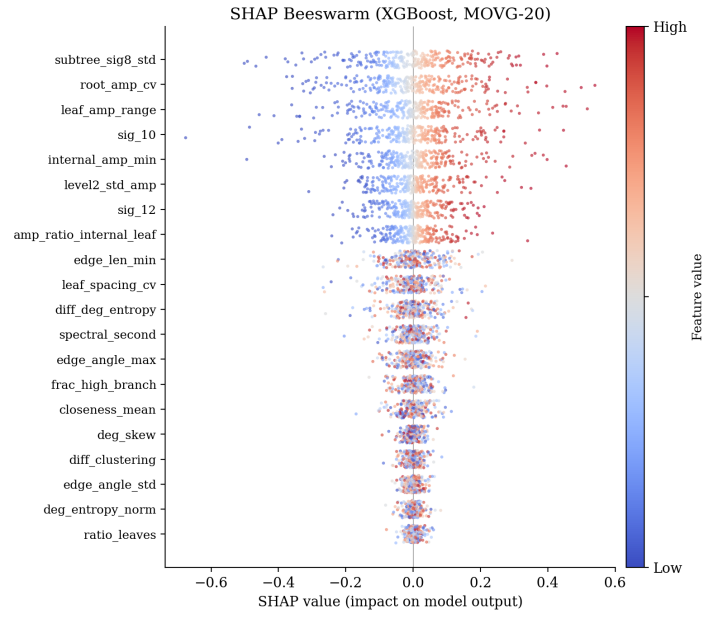


Figure 17: SHAP beeswarm plot for VIX Close predictions

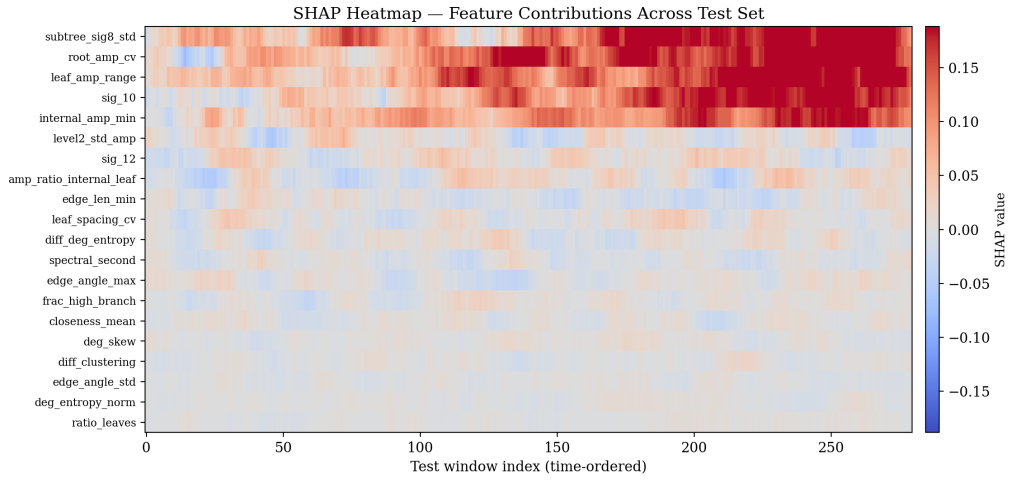


Figure 18: SHAP heatmap for VIX Close predictions

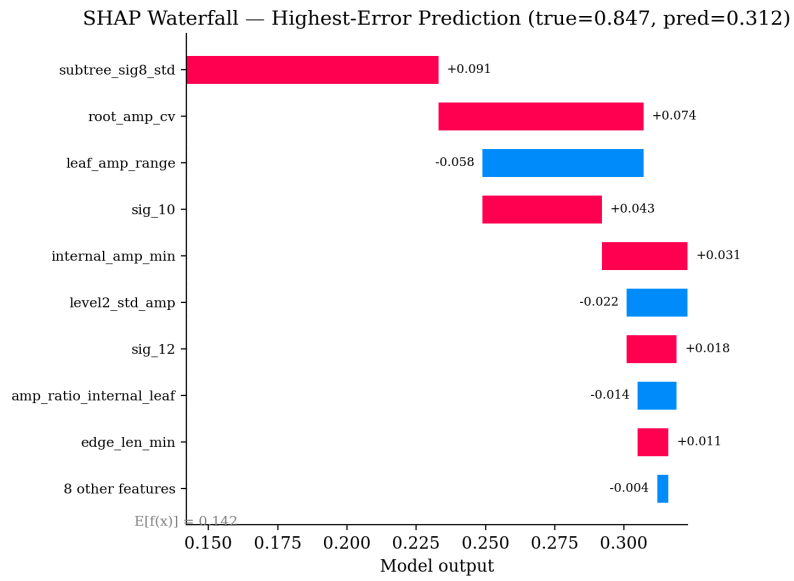


Figure 19: SHAP waterfall plot for a VIX Close prediction

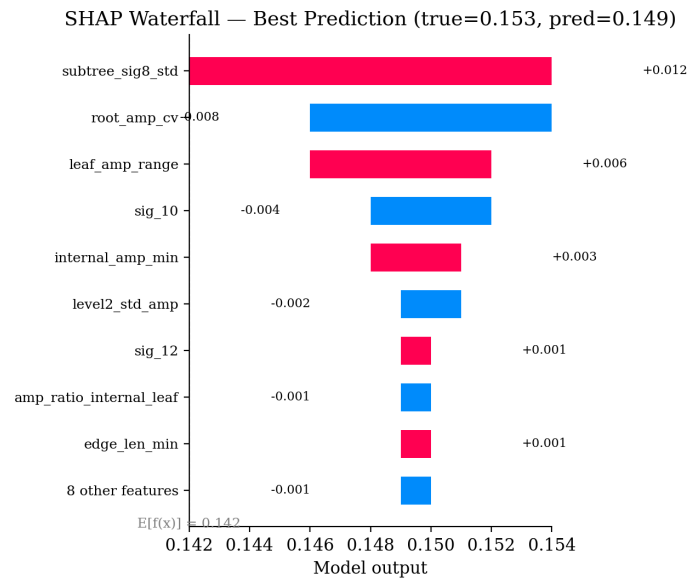


Figure 20: SHAP waterfall plot for a VIX Close prediction

```

    [
      42,
      "root"
    ]
  ],
  "signatures": {
    "first_order": 0.7682506293058395,
    "second_order": 19.944388419389725,
    "third_order": 265.9435886392991
  },
  "sig_scalars": {
    "sig1": 0.7682506293058395,
    "sig2": 19.944388419389725,
    "sig3": 265.9435886392991
  },
  "contribution": 4.452903345097564,
  "rank": 11
},
{
  "chain_id": "chain_008_inc",
  "direction": "increasing",
  "node_ids": [
    13,
    15,
    19,
    34,
    37,
    38,
    39,
    42
  ],
}

```

Figure 21: JSON output from backtrace, showing 1st, 2nd and 3rd order chains

```

.....
-----
Ran 45 tests in 1.932s

OK
PS C:\Users\c3005789\MOVGTest>

```

Figure 22: Unit tests passed, showing that the MOVG is consistent and bug free (for now)

9.2 Reduced Feature Set

The 32 features retained after the three-stage reduction pipeline are listed in Table 2.

9.3 Per-Dataset RMSE Tables

Table 2: The 32 MOVG features produced by `extract_features()`, grouped by category.

Category	Feature name	Description
Core	<code>tree_depth</code>	Maximum hierarchy level
Core	<code>n_edges</code>	Number of visibility edges
Core	<code>avg_degree</code>	Mean node degree
Core	<code>n_decreasing_chains</code>	Total decreasing chains across sampled decompositions
Core	<code>n_increasing_chains</code>	Total increasing chains across sampled decompositions
Core	<code>avg_chain_length</code>	Mean chain length across all decompositions
Core	<code>sig1_mean</code>	Mean first-order signature across chains
Core	<code>sig1_std</code>	Std of first-order signature across chains
Core	<code>sig2_mean</code>	Mean second-order signature across chains
Core	<code>sig2_std</code>	Std of second-order signature across chains
Core	<code>sig3_mean</code>	Mean third-order signature across chains
Core	<code>sig3_std</code>	Std of third-order signature across chains
Tree structure	<code>n_leaves</code>	Number of leaf nodes in hierarchy
Tree structure	<code>mean_branching_factor</code>	Mean children per non-leaf node
Tree structure	<code>root_level_count</code>	Nodes at hierarchy level 1 (root's direct children)
Amplitude	<code>root_amp</code>	Amplitude of root node
Amplitude	<code>leaf_amp_mean</code>	Mean amplitude of leaf nodes
Amplitude	<code>amp_range_tree</code>	Max minus min amplitude over all tree nodes
Degree dist.	<code>degree_max</code>	Maximum node degree
Degree dist.	<code>degree_std</code>	Std of degree distribution
Degree dist.	<code>degree_entropy</code>	Shannon entropy of degree distribution
Spectral	<code>spectral_radius</code>	Largest eigenvalue of Laplacian
Spectral	<code>algebraic_connectivity</code>	Fiedler value (2nd-smallest Laplacian eigenvalue)
Spectral	<code>spectral_gap</code>	Difference between two largest eigenvalues
Centrality	<code>betweenness_mean</code>	Mean betweenness centrality
Centrality	<code>closeness_mean</code>	Mean closeness centrality
Centrality	<code>eigenvector_centrality_max</code>	Maximum eigenvector centrality
Graph-theoretic	<code>clustering_coeff_mean</code>	Mean clustering coefficient
Graph-theoretic	<code>transitivity</code>	Global clustering coefficient
Graph-theoretic	<code>assortativity</code>	Degree assortativity coefficient
Visibility-time	<code>visibility_edge_density</code>	Edge count / max possible edges
Visibility-time	<code>visibility_edge_length_mean</code>	Mean $ i - j $ over edges

Table 3: Walk-forward RMSE on **ETTh1**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	0.204	0.712	0.267
KNN	0.335	0.713	0.276
RF	0.220	0.637	0.258
LGBM	<u>0.201</u>	0.630	0.252
SVR	0.231	0.653	0.268
XGB	–	<u>0.627</u>	<u>0.249</u>

Table 4: Walk-forward RMSE on **ETTh2**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	0.367	0.987	0.436
KNN	0.470	0.996	0.448
RF	0.362	0.943	0.447
LGBM	<u>0.331</u>	0.902	<u>0.421</u>
SVR	0.340	0.928	0.442
XGB	0.333	<u>0.899</u>	0.424

Table 5: Walk-forward RMSE on **EURUSD**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	0.142	0.715	0.261
KNN	0.341	0.759	0.270
RF	0.142	<u>0.665</u>	0.247
LGBM	<u>0.135</u>	0.690	<u>0.237</u>
SVR	0.168	0.685	0.253
XGB	0.138	0.697	0.240

Table 6: Walk-forward RMSE on **SGD/USD**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	<u>0.158</u>	<u>1.494</u>	0.283
KNN	0.383	1.622	0.270
RF	0.186	1.523	0.279
LGBM	0.169	1.521	0.260
SVR	0.251	1.560	0.349
XGB	0.175	1.524	<u>0.259</u>

Table 7: Walk-forward RMSE on **Gold**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	0.075	<u>0.586</u>	0.129
KNN	0.232	0.747	0.135
RF	0.068	0.727	0.126
LGBM	<u>0.066</u>	0.739	0.120
SVR	0.095	0.720	0.139
XGB	0.069	0.741	<u>0.118</u>

Table 8: Walk-forward RMSE on **VIX**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	<u>0.498</u>	0.857	0.678
KNN	0.701	0.919	0.621
RF	0.621	0.851	0.634
LGBM	0.624	<u>0.849</u>	0.641
SVR	0.571	0.878	<u>0.606</u>
XGB	0.602	0.866	0.638

Table 9: Walk-forward RMSE on **Weather**. Best per row in bold; column-best underlined.

Model	MOVG	FFT	Wavelet
Ridge	0.352	<u>0.640</u>	0.471
KNN	0.503	0.736	0.485
RF	0.365	0.662	0.471
LGBM	<u>0.341</u>	0.648	<u>0.458</u>
SVR	0.388	0.685	0.459
XGB	0.358	0.658	0.460